

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 06 -

ESTRATÉGIAS DE DEPLOY AVANÇADAS: CANARY

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	5
SAIBA MAIS.....	6
MERCADO, CASES E TENDÊNCIAS	18
O QUE VOCÊ VIU NESTA AULA?	18
REFERÊNCIAS.....	25

O QUE VEM POR AÍ?

Publicar uma nova versão do software em produção pode ser uma experiência tensa. Mesmo com extensos testes de QA, o tráfego real de produção costuma revelar problemas inesperados. Um deploy direto (“big bang”), atualizando a versão para **100%** dos usuários de uma só vez, é um salto arriscado: qualquer bug ou regressão afetará todo mundo, possivelmente causando downtime, frustração nos usuários e uma corrida caótica para reverter.

As estratégias do tipo “canário” surgem para mitigar esse risco, oferecendo um caminho de rollout mais **cauteloso e controlado**. Em vez de expor todos os usuários à nova versão de imediato, expomos apenas uma pequena fração inicialmente – assim como os canários nas minas de carvão que alertavam os mineradores sobre gases tóxicos. Essa “nova versão” atua como sentinela: se ela apresentar bom desempenho para esse subconjunto, podemos **progredir gradualmente** até o rollout completo; se surgirem falhas, o impacto fica limitado e podemos reverter rapidamente, protegendo a maioria dos usuários.

Em termos práticos, uma **Implantação Canário** (ou *Canary Release*) é uma técnica que reduz o risco ao introduzir uma nova versão em produção **de forma incremental**. Inicialmente, a nova versão é colocada em produção ao lado da versão estável, mas sem receber tráfego, ou seja, nenhum usuário é roteado para ela ainda. Após verificarmos que a nova versão está estável em ambiente real e configurar a infraestrutura (contêineres, pods etc.), começamos a direcionar uma pequena parcela de usuários reais para essa versão canário. Pode-se escolher esses usuários de várias maneiras: aleatoriamente (amostragem), por perfil ou localização (por exemplo, liberar primeiro apenas em uma região ou para usuários internos).

O importante é que, nesse estágio inicial, apenas uma minoria experimenta a nova versão – limitando o **raio de impacto** de eventuais bugs. Monitoramos de perto o comportamento do canário em condições de produção: métricas de erros, latência, uso de recursos e feedback dos usuários são coletados em tempo real. Com isso, obtemos um **feedback seguro e precoce** sobre a atualização.

Se tudo correr bem, ganhamos confiança para aumentar gradualmente a fatia de tráfego direcionada ao canário; se algo sair errado, a reversão é simples – basta

redirecionar os usuários de volta para a versão antiga estável, praticamente sem downtime percebido.

Diferentemente de uma implantação blue-green (em que se alterna todo o tráfego de uma vez entre dois ambientes completos), o deploy canário é **progressivo**. Ele busca limitar ao máximo o impacto de problemas, ao mesmo tempo em que testa a nova versão sob carga real de produção. Essa abordagem permite validação com usuários reais e dados reais antes do rollout completo.

Na prática, a implantação canário aplica o padrão de mudança **paralela**: mantemos duas versões rodando simultaneamente e migramos usuários aos poucos para a nova versão até que, se tudo estiver OK, eventualmente 100% estejam usando-a – momento em que podemos desativar a versão antiga. Por isso a estratégia canário transforma o *deploy* em um processo orientado por **feedback contínuo**: lança-se em pedaços, mede-se resultados, e decide-se promover ou abortar com base em dados. Assim, novos releases deixam de ser saltos no escuro e passam a ser evoluções seguras, aumentando a confiabilidade e a confiança da equipe em cada publicação.

HANDS ON

Vamos agora colocar a “mão na massa” com um exemplo prático de deploy canário em Kubernetes, utilizando **Rust** para orquestrar as etapas. Suponha que temos uma aplicação chamada `myapp` rodando em produção na versão `v1` (estável). Nosso objetivo é lançar a versão `v2` como canário, enviando apenas parte do tráfego para ela e, com base nas métricas coletadas, decidir se promovemos `v2` a estável ou fazemos rollback.

Utilizaremos a `cratek8s` (`clientRust` para Kubernetes) para interagir com o cluster, demonstrando como criar o **Deployment** canário, ajustar a divisão de tráfego via Istio e inspecionar métricas para automatizar decisões.

SAIBA MAIS

Vamos iniciar explicando o passo a passo da aula que vocês verão em vídeo: “Lançando a versão canário (v2) paralela à atual (v1)”.

No Kubernetes, podemos implantar a nova versão v2 criando um novo Deployment com os pods atualizados. No código abaixo, utilizamos kube para definir e criar o Deployment da versão canário. Note que especificamos um rótulo version: v2 para distinguir os pods novos, enquanto os pods da versão estável têm rótulo version: v1. Isso permitirá rotular tráfego mais adiante. Usamos a abordagem JSON para montar o objeto do Deployment, por simplicidade:

```
usekubernetes::Client, Api};
usekubernetes::api::PostParams};
usek8s_openapi::api::apps::v1::Deployment;
useserde_json::json;

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let client = Client::try_default().await?;
    let deploy_api = Api::default_namespaced(client);

    // Definir Deployment da versão canário v2
    let deploy_json = json!({
        "apiVersion": "apps/v1",
        "kind": "Deployment",
        "metadata": { "name": "myapp-v2" },
        "spec": {
            "replicas": 2,
            "selector": { "matchLabels": { "app": "myapp", "version": "v2" } },
            "template": {
                "metadata": { "labels": { "app": "myapp", "version": "v2" } },
                "spec": {
                    "containers": [{
                        "name": "myapp",
                        "image": "myregistry/myapp:v2"
                    }]
                }
            }
        }
    });
    let deploy: Deployment = serde_json::from_value(deploy_json)?;
    deploy_api.create(&PostParams::default(), &deploy).await?;
    println!("Canary Deployment v2 criado com sucesso!");
    Ok(())
}
```

Código-fonte 1 – canary_deployment.rs
Fonte: elaborado pelo autor (2025)

Este código se conecta ao cluster Kubernetes (assumindo credenciais locais em `~/.kube/config`) e então monta um objeto Deployment em memória representando a versão v2 do myapp.

Ao chamar `deploy_api.create(...)`, submetemos esse Deployment ao cluster. A nova versão é implantada em paralelo à v1: teremos pods rodando v1 (digamos, rótulo `version: v1`) e pods rodando v2 (`version: v2`). Note que ainda **não redirecionamos tráfego** para v2 – inicialmente nenhum usuário a está acessando, tal como preconiza a estratégia canário. Nesse estado, podemos fazer smoke tests internos na v2, validar logs, e outras operações de monitoração, enquanto todo tráfego real segue indo para v1.

Dividindo o tráfego entre v1 e v2 com Istio

Para o roteamento condicional, utilizaremos o Istio, que fornece um mesh de serviço com regras de tráfego avançadas. No Istio, definimos um `VirtualService` associado ao host do nosso serviço e um `DestinationRule` que nomeia subsets correspondentes a cada versão (por exemplo, subset “v1” aponta para pods com label `version=v1`, e subset “v2” para `version=v2`). Assim, podemos especificar percentuais de tráfego para cada subset. Por exemplo, poderíamos configurar que **90%** das requisições vão para v1 e **10%** para v2. A modificação desses percentuais pode ser feita dinamicamente via API. A seguir, um trecho de código Rust que demonstra como alterar o `VirtualService` para direcionar 50% do tráfego ao canário (v2) e 50% ao estável (v1). Usamos `DynamicObject` do kube já que o `VirtualService` é um recurso customizado do Istio (CRD):

```
usekube::api::{Api, PatchParams, Patch};
usekube::core::DynamicObject;
useserde_json::json;

// ... (após criar v2, obter cliente)
let vs_api: Api<DynamicObject> = Api::default_namespaced(client);
let traffic_split = json!({
    "spec": {
        "http": [{
            "route": [
                { "destination": { "host": "myapp-service", "subset": "v1" }, "weight": 50 },
                { "destination": { "host": "myapp-service", "subset": "v2" }, "weight": 50 }
            ]
        }
    ]
});
vs_api.patch("myapp-virtualservice", &PatchParams::default(),
    &Patch::Merge(&traffic_split)).await?;
println!("VirtualService atualizado: tráfego 50/50 entre v1 e v2.");
```

Código-fonte 2 – update_virtualservice.rs
Fonte: elaborado pelo autor (2025)

Nesse código, assumimos que já existe um VirtualService chamado myapp-virtualservice, responsável por gerenciar o tráfego do serviço. Aplicamos um **Merge Patch** JSON para alterar sua especificação: agora, as rotas estão divididas em 50% para cada subset/version. Originalmente, começaríamos com algo como 100% v1 / 0% v2; depois de validar a v2 “ociosa”, poderíamos editar para 90/10, em seguida 70/30, etc. – em implementações reais adota-se incrementos logarítmicos (1%, 5%, 10%, 25%, 50%, 100%) ou lineares, dependendo da confiança e do tamanho da base de usuários.

Ferramentas como Istio e Linkerd permitem esse controle fino de tráfego independente do número de réplicas de pods, usando proxies sidecar para inspecionar requisições e aplicar as porcentagens exatas. Alternativamente, poderíamos usar um ingress controller com recurso canário (como NGINX Ingress com anotação de peso) ou um operador específico de canary (como Argo Rollouts) – o conceito é similar: definir fatias de tráfego para cada versão.

MONITORANDO INDICADORES PARA DECIDIR PROMOÇÃO OU ROLLBACK

Com ambas as versões recebendo tráfego (mesmo que inicialmente apenas 5% ou 10% na v2), é fundamental realizar o monitoramento contínuo de **métricas** em tempo real. Indicadores típicos incluem: taxa de erro (códigos 5xx/4xx), latência de requisições (p95, p99), uso de CPU/memória, além de métricas de negócio (conversões, cliques). No nosso cenário, poderíamos integrar uma ferramenta de monitoramento (Prometheus, Datadog etc.) e consultar as métricas via API. Por exemplo, usando a crate `reqwest`, poderíamos consultar o Prometheus para obter a taxa de erro da v2:

```
asyncfncheck_metrics() ->bool {
    // (Pseudo-código de consulta de métrica de erro)
    letresp=reqwest::get("<URL
Prometheus>/api/v1/query?query=rate(myapp_errors{version=\"v2\"}[5m])).await?;
    letresult=resp.json:::<PromQueryResult>().await?;
    leterror_rate=result.data.result[0].value;
    // Retorna true se taxa de erro da v2 for aceitável (abaixo do threshold)
    Ok(error_rate<0.01)
}

// ... em algum loop de rollout:
ifcheck_metrics().await?{
    println!("Métricas OK, promovendo canário para 30% do tráfego.");
    // aumentar peso da v2, e.g., de 10% para 30%
    vs_api.patch("myapp-virtualservice", &PatchParams::default(),
    &Patch::Merge(new_weights)).await?;
} else {
    eprintln!("Métricas ruins, iniciando rollback imediato!");
    // redireciona tráfego de volta 100% para v1
    vs_api.patch("myapp-virtualservice", &PatchParams::default(),
    &Patch::Merge(stable_weights)).await?;
    // opcional: remover Deployment v2
    deploy_api.delete("myapp-v2", &DeleteParams::default()).await?;
}
```

Código-fonte 3 – canary_rollout.rs
Fonte: elaborado pelo autor (2025)

Acima, a função `check_metrics()` representa uma verificação simplificada que consultaria o sistema de monitoramento para verificar a taxa de erro do canário nos últimos 5 minutos, por exemplo. Se a métrica estiver dentro dos critérios de sucesso (digamos, “erro < 1%” ou “não excede a taxa da v1 em mais que 0,1%”), o código prossegue a promoção: aqui ilustrado por um patch que aumentaria o peso de v2 para 30%. Caso a métrica indique degradação (erros ou latência alta), executamos o

rollback: retornamos o tráfego todo para v1 e potencialmente removemos/diminuímos o Deployment v2, abortando o rollout.

Esse tipo de automação fecha o ciclo de realimentação: seu pipeline de deploy pode rodar em loop, aumentando gradativamente o tráfego para o canário enquanto os **dados** suportam a mudança, ou revertendo assim que algum indicador viola os critérios definidos (gatilhos de SLO, por exemplo).

INTEGRAÇÃO VIA KUBE-RS E CONTROLLERS

No exemplo anterior, controlamos o processo manualmente via código. No entanto, em ambientes de produção, é comum utilizar **controladores especializados** para automatizar esse fluxo. Por exemplo, o **Argo Rollouts** que define um *controller* customizado que gerencia o Deployment canário e automatiza as transições de tráfego conforme passos pré-configurados (incluindo consultas a métricas de Prometheus via Analysis Templates). Você poderia, via Rust, interagir com os objetos do Argo Rollouts (CRD `Rollout`) em vez de `VirtualService` diretamente – por exemplo, alterando um campo de peso ou pausando/retomando o rollout programaticamente.

Da mesma forma, ferramentas como o **Flagger** (da Weaveworks) integram-se com service meshes e fazem análise automática de métricas, promovendo ou revertendo conforme resultados estatísticos. Esses operadores estendem o Kubernetes com recursos próprios para progressive delivery, eliminando a necessidade de escrever toda a lógica de decisão manualmente. Ainda assim, compreender o processo subjacente – como fizemos passo a passo – é essencial para ajustar e confiar nessas automações.

A seguir, vamos aprofundar os conceitos teóricos por trás de testes canário e tomada de decisão progressiva, cobrindo fundamentos estatísticos e algoritmos avançados.

TESTES A/B E TESTES SEQUENCIAIS — DEFINIÇÃO MATEMÁTICA

A estratégia canário pode ser vista como uma forma de **experimento A/B** contínuo entre duas versões (A = versão antiga, B = versão nova). Formalmente, temos duas populações de usuários recebendo experiências diferentes, e queremos

testar a hipótese de que a nova versão B é pelo menos tão boa (ou melhor) que A em termos de algum KPI (por exemplo, taxa de erro ou conversão). Definimos hipóteses estatísticas: a **Hipótese Nula (H_0)** afirma que não há diferença significativa entre B e A (ou que B é pior), e a **Hipótese Alternativa (H_1)** afirma que B traz melhoria (por exemplo, menor taxa de erro).

Durante o canário, coletamos dados de métricas de ambas as versões e podemos aplicar testes estatísticos clássicos para avaliar H_0 vs H_1 . Por exemplo, se medirmos proporções de erros, poderíamos usar um teste z de proporções ou um teste qui-quadrado para comparar as taxas de erro de v1 vs v2; para métricas contínuas como latência média, poderíamos usar teste t ou teste não-paramétrico (Mann-Whitney) caso os dados não sejam normais.

O objetivo é determinar se as diferenças observadas ultrapassam um limiar de significância estatística pré-definido (e.g. $\alpha = 0,05$ para erro do Tipo I). Um teste A/B clássico exige tamanho de amostra fixo, calculado com poder estatístico desejado ($1 - \beta$) e efeito mínimo detectável – fórmulas da estatística inferencial fornecem quantas amostras (usuários, requisições) são necessárias para detectar, por exemplo, uma melhora de 5% na métrica com 95% de confiança.

No contexto de deploy canário, entretanto, não queremos esperar coletar **toda** a amostra antes de agir; preferimos avaliar continuamente e **parar cedo** se possível. É aí que entram os **testes sequenciais**. Um teste sequencial permite analisar os dados **em tempo real**, durante a execução do experimento, decidindo antecipadamente critérios de interrupção. Basicamente, após cada pequeno lote de observações, recalculamos a estatística de teste e verificamos se alcançou significância para rejeitar H_0 (sucesso do canário) ou se há evidência forte de falha (canário pior que controle) – em qualquer desses casos, o experimento pode ser encerrado antes do tamanho inicial planejado.

Métodos como o **SPRT (Sequential Probability Ratio Test)** de Wald definem duas barreiras (superior e inferior) de log-verossimilhança cumulativa; quando a estatística ultrapassa a superior, rejeitamos H_0 (isto é, aceitamos que B é melhor), se cai abaixo da inferior, aceitamos H_0 (B não é melhor e possivelmente pior), e se permanece entre as barreiras, continuamos coletando dados.

Assim controlamos os erros α e β de forma sequencial. Outra abordagem recomendada é usar funções de spending de alfa (Alpha Spending) para ajustar o limiar de significância conforme o número de “espiadas” nos dados, evitando aumentar falso-positivos por verificar repetidamente. Assim, testes sequenciais proporcionam **agilidade estatística**: no deploy canário, isso significa que podemos promover ou abortar mais cedo, poupando tempo e reduzindo exposição de usuários se já há evidências suficientes de desempenho (bom ou ruim) da nova versão.

MODELAGEM DE REGRESSÕES E ANÁLISE ESTATÍSTICA DE MÉTRICAS

Uma das principais dificuldades ao decidir sobre o canário é distinguir variações aleatórias de diferenças reais de desempenho, seja uma regressão ou uma melhoria. A estatística nos ajuda a modelar isso.

Por exemplo, podemos modelar a taxa de erro como uma variável aleatória Binomial em que cada requisição resulta em **sucesso ou erro**. A versão v1 tem uma taxa de erro p_1 estimada, e v2 tem p_2 . Montamos $H_0: p_2 = p_1$, $H_1: p_2 > p_1$ (caso suspeitemos que a nova versão possa aumentar erros). Conforme chegam resultados (número de requisições e erros em cada versão), podemos construir **intervalos de confiança** para p_1 e p_2 e verificar se há overlap.

Ferramentas automatizadas de canary, como o **Kayenta** (da Netflix), aplicam exatamente esse tipo de análise: para cada métrica analisada, calcula-se se a diferença entre canário e baseline é estatisticamente significativa. No Kayenta, por exemplo, o algoritmo padrão faz uso de testes não-paramétricos (Mann-Whitney U) para comparar distribuições de métricas e construir intervalos de confiança; se o intervalo indicar diferença significativa desfavorável ao canário, isso contribui para um “score” baixo e possível reprovação do canário.

Além dos testes de hipótese, pode-se usar técnicas de **regressão estatística** (no sentido de modelagem). Por exemplo: regressão linear ou modelos de série temporal podem ajustar a métrica ao longo do tempo para cada versão e tentar prever qual seria o valor esperado sem mudança, detectando se a introdução da versão B causou um *shift* significativo.

Outra abordagem são os **Testes de mudança (change-point detection)** que detectam automaticamente se houve mudança de distribuição após certo ponto (quando iniciamos o tráfego para v2). Assim sendo, a análise estatística robusta é o núcleo para decidir se um comportamento observado com X% dos usuários no canário representa uma verdadeira **regressão** de performance ou apenas ruído aleatório.

TEORIA DE FILAS PARA AVALIAR LATÊNCIA COM TRÁFEGO PARCIAL

Quando dividimos o tráfego entre duas versões, especialmente em sistemas web, estamos, na prática, repartindo a chegada de requisições entre dois conjuntos de servidores/pods. Podemos usar modelos de **teoria das filas** para prever o impacto disso na latência e throughput percebidos.

Imagine que cada versão atende requisições com certa taxa de serviço (μ requests/segundo por pod, e λ é a taxa de chegada de requisições totais do sistema). Se roteamos uma fração φ do tráfego para o canário, então o canário recebe aproximadamente $\lambda^2 = \varphi \times \lambda$ requisições/seg, enquanto a versão estável recebe $\lambda_1 = (1 - \varphi)\lambda$.

Suponha, para simplificar, que cada grupo de pods forma uma fila $M|M|\infty$ (infinitos servidores) ou $M|M|m$ (m servidores). No caso mais simples $M|M|1$ (um servidor por fila), sabemos que o tempo médio de resposta (W) é dado por

$$W = \frac{1}{\mu - \lambda}$$

Com $\lambda < \mu$. Isso implica que, quanto mais próximo a taxa de chegada λ chega da capacidade μ , a latência média cresce de forma não-linear (tendendo ao infinito no ponto de saturação). Quando $\lambda > \mu$, a taxa de chegada de requisições ao serviço é maior do que a taxa de processamento, então o cenário é de filas, saturação e indisponibilidade, portanto inválido.

Com duas versões, se a nova versão tem uma capacidade um pouco menor ($\mu^2 < \mu^1$), então conforme aumentamos φ , mesmo que λ^2 seja pequeno inicialmente, a utilização $\rho_2 = \frac{\lambda_2}{\mu_2}$ pode subir a patamares maiores que $\rho_1 = \frac{\lambda_1}{\mu_1}$, levando a tempos de resposta mais altos para usuários direcionados ao canário. Em outras palavras,

mesmo um canário com 10% do tráfego pode apresentar latência significativamente pior se aquele 10% quase saturar os pods v2 devido a alguma ineficiência introduzida.

A teoria de filas nos permite quantificar isso. Por exemplo, modelando cada pool de pods como um sistema $M|M|m$ (com m pods servidores), podemos estimar a latência média via a fórmula de Erlang-C ou usando a Lei de Little ($L = \lambda W$) para relacionar throughput e tempo na fila. Esses modelos ajudam a prever e dimensionar a fase canário: se a latência esperada do canário a 10% de tráfego já excede um limite (talvez por CPU bound), é um sinal de possível problema de desempenho que só pioraria em 100%.

Em cenários realistas, usamos a fase canário não apenas para medir métricas em proporção menor, mas também para inferir através de escala. Então, se 20% de tráfego na v2 causa 30% de uso de CPU nos pods, podemos extrapolar que 100% de tráfego exigiria ajustar réplicas ou otimizar código, caso contrário a fila de espera cresceria exponencialmente. Assim, a teoria de filas fornece uma base para entender e projetar o comportamento sob carga do rollout progressivo, garantindo que mantemos latência dentro das SLOs conforme ϕ aumenta.

ALGORITMOS DE PROMOÇÃO COM *BANDITS* BAYESIANOS E FUNÇÕES DE CUSTO

Uma abordagem avançada para rollouts canário envolve formular o problema como um *Multi-Armed Bandit*, onde cada versão é uma alternativa com desempenho incerto, e o objetivo é alocar usuários de forma otimizada entre elas. Diferente do teste A/B clássico (onde fixamos metade dos usuários para cada variante até decidir), o algoritmo de bandit *explorac*enários simultâneos, tentando minimizar o custo de expor usuários à pior versão. Em termos práticos, podemos implementar um bandit bayesiano para o canário, onde se envia inicialmente pequenas proporções de tráfego ao novo e ao antigo, e gradualmente, conforme coletamos métricas, aumentamos o tráfego do melhor serviço.

Usando **métodos Bayesianos**, atribuímos distribuições a priori para o desempenho de cada versão (por exemplo, uma distribuição Beta para taxa de sucesso/erro, ou Normal para tempo de resposta) e atualizamos essas distribuições com os dados observados. Um algoritmo popular é o **Thompson Sampling**, que a

cada intervalo amostral sorteia uma probabilidade de sucesso para cada versão a partir de suas distribuições posteriores e direciona o próximo “usuário” para a versão com maior valor amostrado.

Isso naturalmente equilibra exploração (testar o canário o suficiente para ter certeza) e exploração (favorecer a versão com melhor desempenho conhecido). Ao longo do tempo, o *bandit* convergirá para dar quase todo tráfego à versão superior, mas de forma adaptativa – se em algum momento o canário mostrar métricas ruins, o algoritmo reduzirá sua seleção automaticamente.

Podemos incorporar **funções de custo** personalizadas nesses algoritmos: por exemplo, definir a “recompensa” como negativa se uma requisição resultar em erro ou latência alta, ou incluir um custo fixo por enviar um usuário à versão canário (representando aversão ao risco). Um *bandit cost-aware* então busca minimizar o custo total: isso reflete diretamente o dilema do canário – queremos expor o mínimo possível de usuários a uma versão potencialmente ruim (minimizar custo de erro), mas expor o suficiente para detectar melhorias ou problemas com significância (explorar para ter certeza).

Os métodos Bayesianos facilitam também incluir *priors* de conhecimento: por exemplo, se já esperamos que a nova versão melhore a latência, podemos inicializar a distribuição de latência de B com média menor, e o *bandit* confirmará ou refutará isso com os dados. Em aplicações de produção contínua, há pesquisas combinando *bandits* com **garantias de segurança** – por exemplo, travar o algoritmo se a métrica exceder certo limite de erro (mesmo antes da convergência estatística).

No geral, ver o deploy canário como um problema de decisão sequencial sob incerteza nos abre a porta para algoritmos de **Reforço** e *bandits*, que visam **automatizar a promoção** gradativa de forma ótima. Empresas com alto volume de tráfego às vezes optam por esses métodos ao invés de *thresholds* fixos, pois conseguem reação mais rápida e eficiente a sinais de falha ou sucesso, ajustando o rollout continuamente conforme a resposta dos usuários.

ANÁLISE DE ESTRATÉGIAS DE ROLLOUT COM TOPOLOGIA EXPLÍCITA (GRAFO DE VERSÕES)

Outra forma de encarar o rollout canário é modelá-lo como um **grafo de estados**, onde cada estado representa uma configuração de implantação (por exemplo, Estado₀: 100% v1, Estado₁: 90% v1/10% v2, ..., Estado_n: 0% v1/100% v2). As transições nesse correspondem as ações de aumentar ou diminuir tráfego ou de fazer rollback. Esse grafo de versões permite aplicar técnicas de **planejamento e verificação formal**.

Podemos associar probabilidades de sucesso/falha a cada transição (por exemplo, talvez saibamos historicamente que se o canário passar de 10% para 25% sem incidentes, há 90% de chance de sucesso completo). Com um grafo probabilístico assim, podemos calcular o **custo esperado** de cada trajetória de rollout: caminhos mais curtos (menos etapas) têm menos atraso, mas possivelmente maior risco de pular algum problema; caminhos mais graduais reduzem risco, mas impõem mais tempo e esforço.

Otimizar a **topologia do rollout** significa encontrar um equilíbrio ótimo entre velocidade e segurança. Essa análise remete a problemas de decisão Markoviana ou mesmo árvores de decisão: podemos atribuir funções de utilidade (por exemplo, custo de rollback em determinado ponto, custo de demora em promoção) e utilizar **programação dinâmica** ou busca heurística para planejar o melhor grafo de rollout para um determinado sistema.

Em sistemas distribuídos grandes, pode haver grafos de versão mais complexos – por exemplo, vários serviços interdependentes sendo atualizados em canário ao mesmo tempo, formando um grafo multidimensional de estados (combinações de versões em diferentes microserviços).

Nesses casos, técnicas de **exploração de espaço de estados** podem ser aplicadas para garantir que certas combinações perigosas não ocorram simultaneamente, ou para identificar a ordem ótima de implementação canária de múltiplos componentes. Embora esse seja um terreno avançado e ainda em pesquisa, a ideia central é tornar **explícito o espaço de rollout** (todos os passos possíveis) e então analisar propriedades desejadas: “Qual a probabilidade de sucesso final do rollout dado nosso monitoramento e critérios?”, “Qual o pior-case de usuários

impactados se seguirmos esta sequência de porcentagens?”, ou “Existe um ciclo de rollout-rollbacks possíveis que indica instabilidade na versão?”.

Ferramentas de model checking poderiam até simular métricas seguindo certos modelos estatísticos e verificar se a política de rollout atende a certas garantias (por exemplo: pelo menos 99% dos usuários totais verão apenas versões consideradas estáveis). Em resumo, pensar o canário como um grafo de versões e transições nos permite aplicar tanto **teoria de grafos** quanto métodos probabilísticos para tornar o processo mais rigoroso, previsível e otimizado conforme critérios formais de qualidade.

MERCADO, CASES E TENDÊNCIAS

A adoção de deploys canário tornou-se comum em empresas de tecnologia de ponta, justamente por equilibrar inovação rápida e confiabilidade. Vamos ver casos reais (e um fictício) que ilustram práticas de mercado, além de discutir tendências futuras.

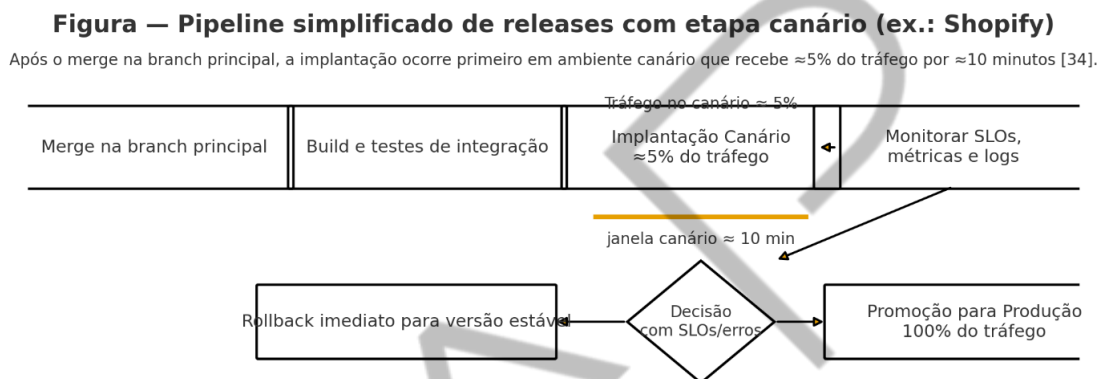


Figura 1 - Pipeline simplificado de releases com a etapa “canário”
Fonte: elaborado pelo autor (2025)

Acima, o processo de deploy do Shopify: após o código ser mesclado na branch principal, ele é implantado primeiro em um ambiente Canary que recebe $\sim 5\%$ do tráfego por ~ 10 minutos.

Nesse período, engenheiros monitoram métricas e realizam testes adicionais. Se nenhum alerta for acionado pela análise automática de canário (que verifica erros e SLOs), então a mudança é promovida diretamente para produção global (100% tráfego). Essa cultura de “limitar o blast radius” está enraizada – o Shopify confia em automações para garantir que qualquer mudança potencialmente problemática afete no máximo alguns usuários antes de ser detida.

Com isso, os desenvolvedores podem implantar dezenas de vezes ao dia com confiança, sabendo que a plataforma atua como uma rede de segurança. Esse caso real mostra como um canário bem integrado ao pipeline CI/CD permite combinar velocidade e segurança*: no Shopify não há janelas fixas ou gerentes de release, cada

equipe desenvolvedora é responsável por suas deploys e confia no canário para proteger a experiência dos milhões de lojistas e compradores.

Outro exemplo emblemático é a Netflix. Conhecida pelo lema “**Testing in Production**”, a Netflix foi além do canário manual e desenvolveu o **Kayenta**, uma ferramenta de *Automated Canary Analysis (ACA)*. Quando a Netflix implementa uma nova funcionalidade, ela é lançada em um subconjunto pequeno de instâncias do serviço, similar ao canário tradicional, porém a avaliação é feita por algoritmos.

O Kayenta compara a versão canário com a baseline (por exemplo, comparando métricas de duas conjuntos de instâncias executando versões diferentes) e realiza uma bateria de testes estatísticos para decidir aprovação. Ele calcula um *score* agregando várias métricas – erro, latência, uso de memória, etc. – atribuindo penalidades se o canário estiver estatisticamente pior em alguma delas.

Se o score mínimo não for atingido, o deploy é abortado automaticamente. A Netflix integrou isso no Spinnaker (sua plataforma de deploy), tornando o processo hands-off: cada mudança de código aciona pipelines que incluem fases canário totalmente automáticas. Esse caso ilustra uma tendência clara: **inteligência artificial e automação nas decisões de rollout**.

Em vez de depender de humanos olhando gráficos, define-se critérios formais e deixa-se a máquina julgar. Não por acaso, empresas como Google e Microsoft também adotaram abordagens similares – o Google SRE Workbook inclusive define canarying como um processo de **parcial deploy + avaliação**, enfatizando que efetivamente é um experimento A/B de produção. Na Netflix e outras *Big Techs* o canário virou um “gate” automatizado: só promove para 100% se passar nos testes estatísticos e nos limites de confiança configurados.

Agora, um exemplo fictício para ilustrar os riscos mitigados pelo canário: imagine a **FinTech Fictícia X** lançando uma nova versão do módulo de transferência bancária. Sem canário, ela faria o deploy para todos os usuários numa madrugada. Apesar dos testes, um caso de erro não previsto derruba transfers internacionais – pela manhã, milhares de clientes frustrados e suporte em chamadas.

Com canário, esse bug teria aparecido para talvez 1% dos usuários durante a madrugada, disparando alarmes quando a taxa de erro da API de transferência subisse acima do normal. O rollout seria automaticamente pausado, engenheiros

seriam alertados durante a janela de canário, e a reversão a tempo evitaria impacto na massa de clientes. Esse cenário hipotético reflete o que empresas do mercado financeiro real (bancos digitais, corretoras) fazem para cumprir seus rigorosos SLOs: todas as mudanças passam por fases graduais, às vezes começando com **canários internos** (ex.: funcionários do banco usando a nova versão primeiro) antes de qualquer cliente final ver algo.

Do lado de empresas brasileiras, podemos citar o Nubank. O Nubank cresceu sua base para dezenas de milhões de clientes e mais de 500 engenheiros, adotando Kubernetes e práticas de implantação moderna para escalar.

Eles mencionam explicitamente o uso de **feature flags e canary releases** para entregar mudanças incrementais e reversíveis, garantindo que funcionalidades novas possam ser desligadas ou revertidas facilmente se algo der errado. Em um estudo de caso da CNCF, o diretor de engenharia do Nubank, Renan Capaverde, destaca que com Kubernetes e deploys canário ficou **mais fácil reverter mudanças e mais rápido implantar**, o que aumentou a frequência de deploys e a confiança das equipes: *“Com Kubernetes e canary deployments, é mais fácil reverter uma mudança, porque também é mais rápido implantar. As pessoas estão entregando com mais frequência e com mais confiança.”*

Hoje, o Nubank realiza centenas de deploys por dia, usando automação pesada. Isso mostra como a cultura de canário suporta o hiper crescimento: permite escalar a engenharia sem medo de implantar, já que qualquer deslizamento atingirá apenas uma fração pequena antes de ser corrigido.

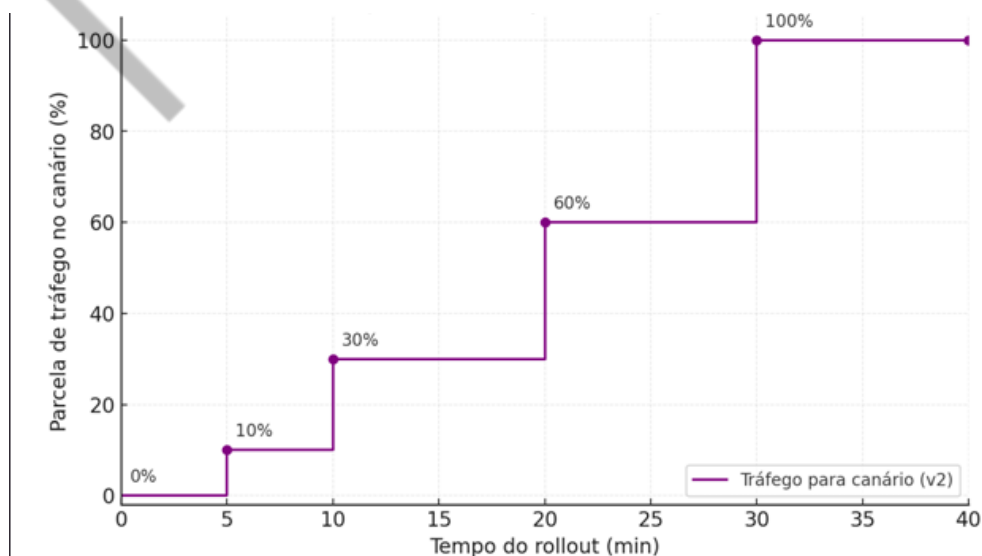


Figura 2 - Distribuição de tráfego ao longo de um rollout “canário”

Fonte: Elaborado pelo autor (2025)

Na linha roxa vemos a parcela de tráfego direcionada à nova versão (canário) ao longo do tempo. Iniciando em 0%, o rollout progride em etapas configuradas (por exemplo, 10% após validar sanidade básica, depois 30%, 60% e finalmente 100% após completa aprovação). Essa progressão **gradual** (neste caso simulada ao longo de ~50 minutos) ilustra o princípio de exposição controlada: cada etapa fornece um intervalo para coletar métricas e confirmar se o sistema está saudável antes de avançar. Se a qualquer ponto fosse detectado um problema severo, o tráfego seria revertido a 0% (rollback) imediatamente, evitando chegar aos 100%.

Na prática, os times definem tanto os incrementos de porcentagem quanto a duração mínima em cada etapa (por exemplo, “10% por 5 minutos, depois 25% por 15 minutos, etc.”) conforme a criticidade da aplicação e o volume de usuários necessário para significância estatística dos indicadores:

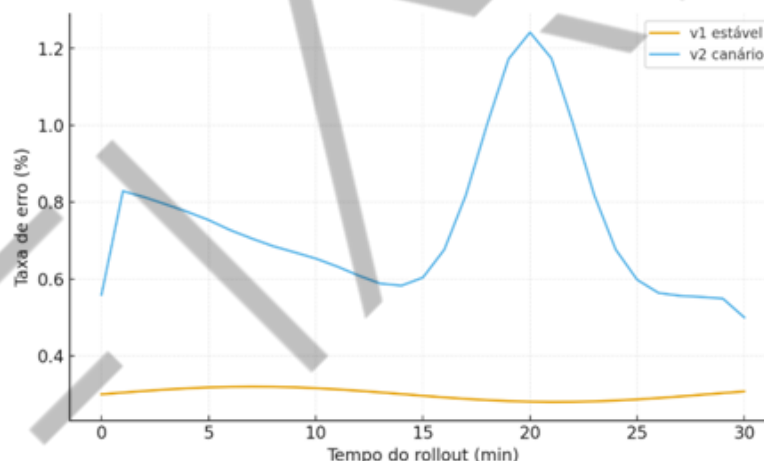


Figura 3 - Taxa de erro comparativa entre a versão estável (v1) e canário (v2) durante o rollout
Fonte: elaborado pelo autor (2025)

As linhas mostram a porcentagem de erros de cada versão ao longo do tempo. Observamos que a versão estável (amarela) mantém ~0.1% de erros, enquanto a versão canário (azul) começa ligeiramente pior (~0.6% de erros), com oscilações no tempo de rollout e convergência para uma taxa de ~0.5%. Esse comportamento poderia indicar, por exemplo, que a nova versão exigiu alguns ajustes inicial (talvez configuracional) mas acabou se mostrando mais eficiente que a antiga.

Em cenários reais, padrões como este seriam altamente positivos – a equipe de engenharia veria que a **versão v2 tem menos erros que v1**, reforçando a decisão

de promovê-la. Por outro lado, se a linha laranja tivesse subido acima da azul de forma consistente, seria um sinal de regressão: o deploy canário seria travado ou revertido antes de alcançar 100%. Empresas como a **Shopify** integram gráficos similares em seus painéis internos: cada deploy mostra lado a lado métricas das versões old vs new, e um alerta é acionado se a diferença ultrapassa um limiar. Assim, o gráfico acima exemplifica a visualização fundamental durante um canário – é uma corrida entre versões, onde a nova precisa provar não ficar para trás (ou idealmente, ser melhor).

FUTURO DO CANARY – GITOPS, SLOS E AI OPS

Olhando adiante, as estratégias canário tendem a se fundir cada vez mais com fluxos **GitOps** e técnicas de inteligência artificial. No modelo GitOps, todo rollout (inclusive o canário) é representado como estado desejado em repositório; imagina-se um futuro em que um pull request possa não só alterar a versão de uma aplicação, mas também incluir *policy* de canário (percentuais, métricas aceitáveis) como código.

Assim, o merge do PR dispararia pipelines declarativos que executam o canário conforme especificado, e controladores reconciliarem o estado (por exemplo, usando ArgoCD e Argo Rollouts de forma uníssona). Isso traz auditabilidade e reprodutibilidade: cada mudança sabe-se como foi lançada e com quais critérios. Outra tendência é alinhar canários diretamente com **SLOs (Service Level Objectives)**.

Em vez de métricas arbitrárias, utiliza-se os mesmos indicadores que definem a saúde do serviço para guiar o rollout. Por exemplo, se o SLO de erro é 0.1%, o canário automaticamente freia se o erro do canário violar essa meta – tornando indistinguível o monitoramento de produção do monitoramento de implantação. Já a Inteligência Artificial pode entrar de diversos modos: algoritmos de detecção de anomalia alimentados por aprendizado de máquina podem identificar padrões sutis nos sinais do canário (talvez correlações complexas entre diversas métricas) e reagir mais rapidamente que regras estáticas.

Além disso, técnicas de **reinforcement learning** podem ser empregadas para ajustar dinamicamente a velocidade do rollout – acelerando se a confiança for alta, ou saltando etapas quando houver alta convicção estatística de sucesso, ou ainda

retrocedendo prudentemente diante de incerteza. Em ambientes de microserviços altamente interconectados, a AI também pode ajudar a **dimensionar experimentos canário**, escolhendo automaticamente um subconjunto representativo de usuários ou tráfego que maximize informação obtida com mínimo impacto.

Algumas empresas estão explorando o conceito de “*canary-as-a-service*” onde uma camada de AI Ops gerencia deploys em múltiplos clusters e regiões, aprendendo com históricos de rollout quais tipos de mudanças costumam falhar e ajustando automaticamente as políticas (por exemplo, uma alteração muito ampla pode automaticamente começar em 1% em vez de 5%, se o modelo de ML prever risco). Em resumo, o canário do futuro tende a ser **mais automatizado, autônomo e inteligente**, reduzindo ainda mais a necessidade de intervenção humana e aumentando a confiabilidade do processo de entrega contínua.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos profundamente a estratégia de deploy canário, destacando sua importância para atualizar sistemas complexos com segurança ao liberar versões novas para pequenos grupos de usuários, colhendo feedback real e protegendo a experiência da maioria.

Vimos um exemplo prático usando Rust, Kubernetes e Istio para criar deployments canários, dividir tráfego, monitorar métricas e automatizar decisões de promoção ou rollback. Abordamos conceitos avançados como testes A/B sequenciais, modelos de filas para prever impacto em performance e algoritmos multi-armed bandits bayesianos para otimizar o rollout adaptativo.

Além disso, discutimos a modelagem do rollout como um grafo de estados para racionalizar e otimizar transições, e analisamos casos reais de empresas como Shopify, Netflix e Nubank, que aplicam canários para garantir agilidade sem comprometer estabilidade.

Por fim, abordamos tendências como GitOps, uso de SLOs para abortar canários e o papel crescente da inteligência artificial em decisões de deploy.

Você sai com uma visão completa do deploy canário — não apenas como um tipo de release, mas como uma filosofia de engenharia orientada a dados e segurança, pronta para elevar a qualidade e confiança nas suas entregas em produção.

REFERÊNCIAS

BARLA, N. **Model Deployment Strategies**. Neptune.ai Blog. 2023. Disponível em: <https://neptune.ai/blog/model-deployment-strategies>. Acesso em: 9 set. 2025.

CNCF. **Case Study: Nubank – How Kubernetes empowered Nubank engineers to deploy 700 times a week**. Cloud Native Computing Foundation. 2020. Disponível em: <https://www.cncf.io/case-studies/nubank/>. Acesso em: 9 set. 2025.

FARIAS, E. **O que você precisa saber sobre testes A/B para não pisar na jaca**. Medium – iFood Tech. 2025. Disponível em: <https://medium.com/ifood-tech/testes-ab-2025>. Acesso em: 9 set. 2025.

GRAFF, M.; SANDEN, C. **Automated Canary Analysis at Netflix with Kayenta**. Netflix TechBlog. 2018. Disponível em: <https://netflixtechblog.com/kayenta-automated-canary-analysis>. Acesso em: 9 set. 2025.

GOOGLE SRE WORKBOOK. **Chapter 16 - Canarying Releases**. In: BEYER, B. et al. Site Reliability Engineering Workbook. Sebastopol: O'Reilly, 2018. Disponível em: <https://sre.google/workbook/canarying-releases/>. Acesso em: 9 set. 2025.

LI, J.; NEELY, K.; GEIGER, J. **Software Release Culture at Shopify**. Shopify Engineering Blog. 2020. Disponível em: <https://shopify.engineering/software-release-culture>. Acesso em: 9 set. 2025.

NETDATA. **What Is Canary Deployment And How To Implement It With Kubernetes**. Netdata Academy. 2023. Disponível em: <https://learn.netdata.cloud/docs/canary-deployment>. Acesso em: 9 set. 2025.

SATO, D. **Implantações Canário**. Blog ThoughtWorks Brasil. 2014. Disponível em: <https://www.thoughtworks.com/pt/insights/blog/implantacoes-canario>. Acesso em: 9 set. 2025.

PALAVRAS-CHAVE

Kubernetes. Rollout. Canary.

EMPRE



POSTECH